

Python的MQTT编程

注意：因为使用端口不同时使用的连接方式不同，所以以下分别介绍1883端口和8084端口上的使用方法。

（一）使用1883端口的MQTT使用。

- 本部分的资料来源：<https://www.emqx.com/zh/blog/how-to-use-mqtt-in-python>

如何在 Python3 中使用 MQTT 客户端库 Paho Client



目录

- 引言
- 为什么选择 Paho MQTT Python 客户端？

- [Python MQTT 项目准备](#)
- [准备 MQTT Broker](#)
- [Paho MQTT Python 客户端使用](#)
- [完整代码](#)
- [测试](#)
- [Paho MQTT Python 客户端的常见问题](#)
- [结语](#)

引言

[MQTT](#) 是一种轻量级的消息传输协议，采用发布/订阅模式，使用最少的代码和带宽提供可靠的实时通信。它特别适用于资源有限和低带宽网络的设备，因此在物联网、移动互联网、车联网和电力行业得到了广泛应用。

Python 因其灵活性、易用性和丰富的库而在物联网中被广泛使用。由于能够处理大量数据，Python 非常适合智能家居自动化、环境监测和工业控制。它与微控制器兼容，使其成为开发物联网设备的重要工具。

本文主要介绍如何在 Python 项目中使用 paho-mqtt 客户端，实施与 MQTT 代理之间的连接、订阅、消息传递等功能。

为什么选择 Paho MQTT Python 客户端？

Paho Python 客户端提供了一个支持 [MQTT 5.0](#)、3.1.1 和 3.1 的客户端，适用于 Python 2.7 或 3.x。它还提供了一些辅助函数，使得向 [MQTT 服务器](#) 发布单条消息变得非常简单。

作为 Python 社区中最受欢迎的 MQTT 客户端库，Paho MQTT Python 客户端具有以下优点：

- 开源且得到社区支持；
- 简洁易用的 API，便于连接 MQTT 服务器并进行消息的发布/订阅；
- 支持多种安全机制；
- 积极开发和维护，以适应快速发展的物联网环境。

Python MQTT 项目准备

Python 版本

该项目在 Python 3.11 中开发和测试。请确认您安装了正确的 Python 版本，可以使用以下命令：

```
1 $ python3 --version
2 Python 3.11.8
```

安装 Paho MQTT 客户端

paho-mqtt 在 2024 年 2 月发布了 2.0.0 版本，相比 1.X 版本有一些重要更新。本文主要演示 1.X 版本的代码，同时也会提供 2.0.0 版本的相应代码，供读者选择合适的 paho-mqtt 版本。

有关 2.0.0 版本的详细变更，请参阅文档：[Migrations — Eclipse paho-mqtt documentation](#)

使用 Pip 安装 paho-mqtt 1.X

```
1 pip3 install "paho-mqtt<2.0.0"
```

使用 Pip 安装 paho-mqtt 2.X

```
1 pip3 install paho-mqtt
```

如果您需要安装 Pip 的帮助，请参考官方文档：[Installation – pip documentation v24.2](#)。该资源提供了在不同操作系统和环境安装 Pip 的详细说明。

准备 MQTT Broker

在开始之前，请确保您有一个 [MQTT Broker](#) 用于通信和测试。我们建议使用 EMQX Platform 的 Serverless 版本。

EMQX Platform 是一个全托管的 MQTT 消息云服务，可以无缝连接您的物联网设备到任何云端，无需维护基础设施。EMQX Serverless 在安全、可扩展的集群上提供 MQTT 服务，并采用按量计费的定价模式，是适合快速开启 MQTT 项目的灵活经济的解决方案。

免费试用 EMQX Platform

[开始试用 →](#)

为简化流程，本文将使用[免费的公共 MQTT 服务器](#)：

- 服务器: `http://broker.emqx.io`
- TCP 端口: `1883`
- WebSocket 端口: `8083`
- SSL/TLS 端口: `8883`
- 安全 WebSocket 端口: `8084`

Paho MQTT Python 客户端使用

导入 Paho MQTT 客户端:

```
1 from paho.mqtt import client as mqtt_client
```

创建 MQTT 连接

TCP 连接

我们需要指定 MQTT 连接的代理地址、端口和主题。此外, 我们可以使用 Python 的 `random.randint` 函数生成随机的客户端 ID。

```
1 broker = 'broker.emqx.io'
2 port = 1883
3 topic = "python/mqtt"
4 client_id = f'python-mqtt-{random.randint(0, 1000)}'
5 # username = 'emqx'
6 # password = 'public'
```

了解更多请查看博客: [创建 MQTT 连接时如何设置参数](#)

接下来, 我们需要编写 `on_connect` 回调函数, 以便连接代理。此函数在客户端成功连接后被调用, 我们可以使用 `rc` 参数检查连接状态。通常, 我们还会创建一个同时连接到 `broker.emqx.io` 的客户端对象。

```
1 def connect_mqtt():
2     def on_connect(client, userdata, flags, rc):
3         # For paho-mqtt 2.0.0, you need to add the properties parameter.
4         # def on_connect(client, userdata, flags, rc, properties):
5             if rc == 0:
6                 print("Connected to MQTT Broker!")
7             else:
8                 print("Failed to connect, return code %d\n", rc)
9         # Set Connecting Client ID
10        client = mqtt_client.Client(client_id)
11
12        # For paho-mqtt 2.0.0, you need to set callback_api_version.
13        # client = mqtt_client.Client(client_id=client_id, callback_api_versio
n=mqtt_client.CallbackAPIVersion.VERSION2)
14
15        # client.username_pw_set(username, password)
16        client.on_connect = on_connect
17        client.connect(broker, port)
18        return client
```

自动重连

在 [MQTT 客户端库](#) 中，自动重连功能确保在不稳定的网络条件下，设备与代理之间可靠的通信，无需人工干预。当网络连接中断或代理暂时不可用时，客户端可以恢复发布或订阅主题，这对于汽车系统和医疗设备等高可靠性应用至关重要。

Paho MQTT 客户端的自动重连代码如下：

```
1 FIRST_RECONNECT_DELAY = 1
2 RECONNECT_RATE = 2
3 MAX_RECONNECT_COUNT = 12
4 MAX_RECONNECT_DELAY = 60
5
6 def on_disconnect(client, userdata, rc):
7     logging.info("Disconnected with result code: %s", rc)
8     reconnect_count, reconnect_delay = 0, FIRST_RECONNECT_DELAY
9     while reconnect_count < MAX_RECONNECT_COUNT:
10         logging.info("Reconnecting in %d seconds...", reconnect_delay)
11         time.sleep(reconnect_delay)
12
13         try:
14             client.reconnect()
15             logging.info("Reconnected successfully!")
16             return
17         except Exception as err:
18             logging.error("%s. Reconnect failed. Retrying...", err)
19
20         reconnect_delay *= RECONNECT_RATE
21         reconnect_delay = min(reconnect_delay, MAX_RECONNECT_DELAY)
22         reconnect_count += 1
23     logging.info("Reconnect failed after %s attempts. Exiting...", reconnect_count)
```

然后，将其设置为客户端对象的 `on_disconnect` 。

```
1 client.on_disconnect = on_disconnect
```

客户端自动重连的完整代码请见：[GitHub](#)。

TLS/SSL

在 MQTT 中使用 TLS 可以确保信息的机密性和完整性，防止信息泄露和篡改。TLS 认证可以分为单向认证和双向认证。

单向认证

Paho MQTT 客户端的单向认证代码如下：

```
1 def connect_mqtt():
2     client = mqtt_client.Client(CLIENT_ID)
3     client.tls_set(ca_certs='./broker.emqx.io-ca.crt')
```

双向认证

Paho MQTT 客户端的双向认证代码如下：

```
1 def connect_mqtt():
2     client = mqtt_client.Client(CLIENT_ID)
3     client.tls_set(
4         ca_certs='./server-ca.crt',
5         certfile='./client.crt',
6         keyfile='./client.key'
7     )
```

发布消息

创建一个 while 循环，每秒向主题 `/python/mqtt` 发送一条消息，并在发送 5 条消息后退出循环。

```
1 def publish(client):
2     msg_count = 1
3     while True:
4         time.sleep(1)
5         msg = f"messages: {msg_count}"
6         result = client.publish(topic, msg)
7         # result: [0, 1]
8         status = result[0]
9         if status == 0:
10            print(f"Send `{msg}` to topic `{topic}`")
11        else:
12            print(f"Failed to send message to topic {topic}")
13        msg_count += 1
14        if msg_count > 5:
15            break
```

订阅

创建消息回调函数 `on_message`，当客户端收到来自 MQTT Broker 的消息时触发。我们将在此函数中打印订阅主题的名称和收到的消息。

```
1 def subscribe(client: mqtt_client):
2     def on_message(client, userdata, msg):
3         print(f"Received `{msg.payload.decode()}` from `{msg.topic}` topic")
4
5     client.subscribe(topic)
6     client.on_message = on_message
```

完整代码

MQTT 消息发布代码


```
1  # python 3.11
2
3  import random
4  import time
5
6  from paho.mqtt import client as mqtt_client
7
8
9  broker = 'broker.emqx.io'
10 port = 1883
11 topic = "python/mqtt"
12 # Generate a Client ID with the publish prefix.
13 client_id = f'publish-{random.randint(0, 1000)}'
14 # username = 'emqx'
15 # password = 'public'
16
17 def connect_mqtt():
18     def on_connect(client, userdata, flags, rc):
19         if rc == 0:
20             print("Connected to MQTT Broker!")
21         else:
22             print("Failed to connect, return code %d\n", rc)
23
24     client = mqtt_client.Client(client_id)
25     # client.username_pw_set(username, password)
26     client.on_connect = on_connect
27     client.connect(broker, port)
28     return client
29
30
31 def publish(client):
32     msg_count = 1
33     while True:
34         time.sleep(1)
35         msg = f"messages: {msg_count}"
36         result = client.publish(topic, msg)
37         # result: [0, 1]
38         status = result[0]
39         if status == 0:
40             print(f"Send `{msg}` to topic `{topic}`")
41         else:
42             print(f"Failed to send message to topic {topic}")
43         msg_count += 1
44         if msg_count > 5:
```

```
45             break
46
47
48 def run():
49     client = connect_mqtt()
50     client.loop_start()
51     publish(client)
52     client.loop_stop()
53
54
55 if __name__ == '__main__':
56     run()
```

MQTT 订阅代码

```
1  # python 3.11
2
3  import random
4
5  from paho.mqtt import client as mqtt_client
6
7
8  broker = 'broker.emqx.io'
9  port = 1883
10 topic = "python/mqtt"
11 # Generate a Client ID with the subscribe prefix.
12 client_id = f'subscribe-{random.randint(0, 100)}'
13 # username = 'emqx'
14 # password = 'public'
15
16
17 def connect_mqtt() -> mqtt_client:
18     def on_connect(client, userdata, flags, rc):
19         if rc == 0:
20             print("Connected to MQTT Broker!")
21         else:
22             print("Failed to connect, return code %d\n", rc)
23
24     client = mqtt_client.Client(client_id)
25     # client.username_pw_set(username, password)
26     client.on_connect = on_connect
27     client.connect(broker, port)
28     return client
29
30
31 def subscribe(client: mqtt_client):
32     def on_message(client, userdata, msg):
33         print(f"Received `{msg.payload.decode()}` from `{msg.topic}` topic")
34
35     client.subscribe(topic)
36     client.on_message = on_message
37
38
39 def run():
40     client = connect_mqtt()
41     subscribe(client)
42     client.loop_forever()
43
```

```
44
45 if __name__ == '__main__':
46     run()
```

测试

订阅

运行 MQTT 订阅脚本 `sub.py`，我们将看到客户端成功连接并开始等待发布者发布消息。

```
1 python3 sub.py
```

```
→ ~ python3 sub.py
Connected to MQTT Broker!
```

发布消息

运行 MQTT 消息发布脚本 `pub.py`，我们会看到客户端成功连接并发布了 5 条消息。同时 `sub.py` 也会成功收到 5 条消息。

```
1 python3 pub.py
```

```
→ ~ python3 sub.py
Connected to MQTT Broker!
Received `messages: 1` from `python/mqtt` topic
Received `messages: 2` from `python/mqtt` topic
Received `messages: 3` from `python/mqtt` topic
Received `messages: 4` from `python/mqtt` topic
Received `messages: 5` from `python/mqtt` topic

|→ ~ python3 pub.py
|Connected to MQTT Broker!
|Send `messages: 1` to topic `python/mqtt`
|Send `messages: 2` to topic `python/mqtt`
|Send `messages: 3` to topic `python/mqtt`
|Send `messages: 4` to topic `python/mqtt`
|Send `messages: 5` to topic `python/mqtt`
|→ ~
```

Paho MQTT Python 客户端的常见问题

如果不执行 `loop_stop()` 会发生什么？

`Loop_stop()` 用于停止 MQTT 客户端的消息循环并将其标记为已停止。此过程可确保客户端正常关闭，从而降低消息丢失、连接泄漏和异常程序行为等问题的风险。

例如，在本文提供的 `pub.py` 示例中，删除 `client.loop_stop()` 方法可能会导致 `sub.py` 脚本接收到的消息少于 5 条。

因此，正确使用 `loop_stop()` 方法来确保 MQTT 客户端正常关闭并防止由于未关闭连接而可能出现的任何潜在问题至关重要。

`connect_async()` 是用来做什么的？

`connect_async()` 在 MQTT 客户端应用程序需要长期 MQTT 连接或需要在后台保持 MQTT 连接处于活动状态而不阻塞主线程的情况下很有用。其主要使用场景有：

- **长期 MQTT 连接：** `connect_async()` 有助于防止需要长期 MQTT 连接的 MQTT 客户端应用程序停滞或无响应，例如在工业应用程序中。
- **网络连接不稳定：** 在网络连接不确定或不稳定的环境中，可以使用 `connect_async()` 通过重试和延迟建立连接来提高应用程序的可靠性。
- **频繁的连接和参数更改：** 当连接参数或其他设置频繁更改时，`connect_async()` 有助于提高应用程序响应能力并防止卡顿。
- **后台 MQTT 连接：** `connect_async()` 允许在应用程序运行其他进程时在后台建立 MQTT 连接，从而增强用户体验。

结语

本文介绍了如何使用 `paho-mqtt` 客户端连接到免费的公共 MQTT Broker。我们成功实现了连接过程，使用 `publish()` 方法将消息从测试客户端发送到 Broker，并使用 `subscribe()` 方法从 Broker 订阅消息。

接下来，您可以查看由 EMQ 提供的《[MQTT 教程](#)》系列，了解 MQTT 协议的特性，探索更多 MQTT 的高级应用，进行 MQTT 应用与服务开发。

（二）采用8084端口的使用方法

⚠ 注意：用 8084 端口时，的连接方式与第一部分的1883端口的使用方法不同。

用8084端口连接borker的示例代码

以下是使用 Python 的 Paho MQTT 客户端连接 8084 端口（通常用于 MQTT over WebSocket）的示例代码。假设你连接的是一个支持 WebSocket 的 MQTT 代理（如 Mosquitto 或云服务）：

```
1 import paho.mqtt.client as mqtt
2
3 # 定义回调函数 (根据需求实现)
4 def on_connect(client, userdata, flags, rc):
5     if rc == 0:
6         print("连接成功! ")
7         client.subscribe("your/topic") # 订阅主题
8     else:
9         print(f"连接失败, 错误码: {rc}")
10
11 def on_message(client, userdata, msg):
12     print(f"收到消息: {msg.payload.decode()} 主题: {msg.topic}")
13
14 # 创建客户端实例 (指定使用 WebSocket 传输)
15 client = mqtt.Client(transport="websockets") # 关键参数!
16
17 # 设置 TLS (如果代理使用 wss:// 安全协议)
18 # client.tls_set(ca_certs="path/to/ca.crt") # 根据实际情况提供 CA 证书路径
19 # 或者跳过证书验证 (不推荐生产环境使用)
20 # client.tls_insecure_set(True)
21
22 # 设置回调函数
23 client.on_connect = on_connect
24 client.on_message = on_message
25
26 # 连接参数
27 broker = "your.broker.address" # 替换为实际代理地址
28 port = 8084
29 keepalive = 60
30
31 # 如果需要认证
32 # client.username_pw_set("your_username", "your_password")
33
34 # 发起连接
35 client.connect(broker, port, keepalive)
36
37 # 保持网络循环
38 client.loop_forever() # 或者使用 loop_start() 非阻塞方式
```

关键点解释:

1. `transport="websockets"`

指定使用 WebSocket 传输协议（默认是 TCP）。这是连接 8084 端口的关键，因为该端口通常用于 WebSocket 通信。

2. TLS 配置

- 如果代理使用 `wss://`（WebSocket Secure），需通过 `tls_set()` 启用 SSL/TLS。
- 根据代理要求提供 CA 证书路径，或使用 `tls_insecure_set(True)` 跳过验证（仅测试环境）。

3. 端口号

确保代理确实在 8084 端口监听 WebSocket 连接，常见的组合：

- `ws://` (非加密) → 通常端口 80 或 8083
- `wss://` (加密) → 通常端口 443 或 8084

4. 认证

如果代理需要用户名/密码，使用 `username_pw_set()` 方法设置。

常见问题排查：

• 连接超时

检查代理地址、端口是否正确，防火墙是否放行 8084 端口。

• 证书错误

确认 CA 证书路径正确，或尝试 `tls_insecure_set(True)` 临时排除证书问题。

• 协议不匹配

确保代理端配置支持 WebSocket，而非普通 TCP MQTT。

根据你的实际代理配置调整参数。如果仍有问题，建议查看代理的日志或文档确认连接要求。